8th International Conference on Frontiers in Academic Research

July 16-17, 2026: Konya, Turkey

© 2026 Published by All Sciences Academy

https://www.icfarconf.com/

Gradient at the Edge of Chaos: The Breaking Point of Analytic Gradient Methods in HydroGym

Haktan Gülöğlu*

1Sakarya University, National Technology Workshop, Artificial Intelligence Center, Türkiye

* *(haktanguloglu24@gmail.com) Email of the corresponding author*

Abstract – This work characterizes the breaking point at the edge of chaos — the first training episode at which numerical divergence occurs — for analytic-gradient reinforcement learning on a chaotic fluid-dynamics control task. The Nature paper introducing HydroGym reports that Gradient-Enhanced PPO (GPPO) — which mixes an analytic reward-trajectory gradient, backpropagated through the differentiable fluid solver, with the standard PPO gradient — cuts training iterations on the Kolmogorov-flow environment by at least 65% relative to PPO, shown at a single Reynolds number and seed. We evaluate PPO, GPPO, SAC, and a gradient-enhanced SAC variant (GSAC) on the real hydrogym.jax Kolmogorov-flow environment at a regular (Re=45) and a chaotic (Re=300) point, across two training seeds. At Re=45, GPPO's sample-efficiency advantage reproduces in both seeds (63% and 77% fewer episodes, close to the paper's 65%). At Re=300, all four algorithms eventually diverge (NaN), but the divergence episode differs sharply by algorithm and seed: in seed=0, PPO/SAC/GPPO/GSAC break at episodes 12/36/83/141; in seed=1, at 35/6/40/63 — SAC now breaks earlier than PPO, reversing the seed=0 pattern. Analytic-gradient injection delays divergence (GPPO>PPO, GSAC>SAC) in both seeds, and GSAC is the most robust of the four. The break is a sudden, precursor-free event, and a short post-trigger survival window (seen only for GSAC, seed=0) is quantitatively explained — correctly predicted absent in seed=1 — by a replay-buffer dilution mechanism. We conclude that analytic-gradient injection delays, but does not eliminate, chaotic-regime instability, and that off-policy training only reliably extends this delay when combined with the analytic gradient.

Keywords – reinforcement learning, differentiable simulation, analytic policy gradients, fluid dynamics control, chaotic dynamics, proximal policy optimization, soft actor-critic

I. Introduction

The active control of fluid dynamics — drag reduction, mixing enhancement, extreme-event suppression — has become one of the most active application areas of reinforcement learning (RL) in recent years. The founding study by Rabault et al. [23] first showed that deep RL can achieve an 8% drag reduction at Re=100 by controlling the periodic vortex shedding behind a cylinder; this line of work has since extended into the turbulent regime, including control of a turbulent separation bubble [10] and real-time, state-dependent turbulent wake control [24]. HydroGym [1] unifies these scattered codebases into a single, solver-agnostic platform offering more than 61 environments from laminar to turbulent, and, alongside classical LBM/FEM solvers, includes an end-to-end **differentiable** solver built on JAX [13] — a direct continuation of pioneering differentiable-CFD libraries such as JAX-CFD [14] and JAX-Fluids [15,16] (notably, JAX-Fluids authors Bezgin and Buhendwa are also HydroGym co-authors). FluidGym [2], a methodologically related, fully differentiable, PyTorch-based flow-control benchmark suite, was developed with similar motivation.

Differentiability enables RL to go beyond standard model-free training by computing **analytic policy gradients** — the derivative of reward with respect to policy parameters, obtained by direct backpropagation through the simulator rather than by sampling. The HydroGym paper demonstrates this with GPPO: an analytic gradient is mixed with the standard likelihood-ratio PPO gradient, reducing training iterations on the Kolmogorov-flow environment by at least 65% and producing more efficient controllers with lower action amplitude [1]. This result, however, is shown at a **single operating point**; how the analytic gradient behaves at the regular and chaotic ends of the Re=40–500 range the paper itself defines for Kolmogorov flow is left untested.

This gap overlaps a well-known risk in the differentiable-programming literature. Metz et al. [7] demonstrate a common "chaos-induced failure mode" in iterated maps, numerical physics simulations, and learned optimizers, tracing it to the spectrum of the system Jacobian. On the fluid-dynamics side, Boffetta and Musacchio [25] show that the leading Lyapunov exponent of homogeneous-isotropic turbulence grows with Reynolds number (faster than the dimensional estimate) and that the predictability horizon shrinks inversely with this exponent. Together, these two independent theoretical results — one from optimization, one from fluid dynamics — suggest that in a chaotic regime the norm/direction of the analytic gradient may become hypersensitive through a butterfly-effect mechanism, and that GPPO's reported advantage may weaken, or even reverse, as Re increases.

Analytic-gradient injection has so far been applied mostly with **on-policy** methods (like GPPO) or in rigid-body/contact-rich robotics tasks: SHAC [3] mitigates exploding/vanishing gradients with short-horizon rollouts and a learned terminal value function; AHAC [5] takes this a step further by **adaptively** shrinking the horizon to avoid stiff dynamics rather than keeping it fixed. On the off-policy side, SAPO [4] carries analytic-gradient RL into a maximum-entropy (SAC-like) framework and shows superior performance on deformable-object tasks — but this approach has never been applied to fluid dynamics. This work adds **GSAC**, an analytic-gradient-enhanced version of SAC [8], as a fourth method, carrying the intersection of the GPPO/SHAC family and the SAPO family into fluid dynamics and testing whether off-policy replay-buffer structure tolerates gradient instability in the chaotic regime better than on-policy GPPO.

Contributions:

- We extend HydroGym's GPPO-vs-PPO result on Kolmogorov flow [1] from a single Re point to a regular-vs-chaotic regime comparison.
- We introduce GSAC (SAC + analytic gradient), a variant never previously applied to HydroGym and the first transfer of the SAPO [4] family to fluid dynamics, and for the first time disentangle whether the stability advantage originates from the "analytic gradient" or from "off-policy training structure."
- We show that all four algorithms eventually diverge numerically in the chaotic regime, but that the divergence episode varies substantially with both algorithm and training seed (range: 12–141 episodes). Direct comparison of **two independent seeds** reveals that the "clean multiplicative" pattern observed at n=1 is an artifact: the delaying effect of the analytic gradient is directionally consistent across both seeds (GPPO>PPO, GSAC>SAC), but the claim that off-policy structure **alone** is delaying reverses at seed=1 (SAC collapses earlier than PPO). The one stable finding is that **GSAC is the most robust of the four methods in both seeds**.
- By directly inspecting the raw training curves we characterize the mechanism of the break: there is no visible precursor in the reward/gradient signal before the break (a sudden, stochastic trigger, observed in both seeds); in on-policy methods the trigger coincides with permanent collapse, whereas in off-policy methods dilution of poisoned transitions in the replay buffer can delay permanent collapse by a few episodes — a quantitative prediction derived from this mechanism correctly anticipates **both** the 2-episode delay observed for GSAC in seed=0 **and** the absence of any delay in seed=1 (§III-C).

Hypothesis: the norm/variance of the analytic gradient will be stable at low Re and unstable at high Re; against this instability, GSAC's off-policy structure will remain more robust than GPPO's on-policy structure. **(n=2 outcome:** GSAC's greater robustness relative to GPPO is confirmed in both seeds (seed=0: 141 vs. 83; seed=1: 63 vs. 40); however, the reason is not that "off-policy structure alone is more robust," but that off-policy structure only reliably contributes **together with** the analytic gradient — see §III-B.)

II. Materials and Method

A. Environment

Experiments run on the real `hydrogym.jax.envs.kolmogorov.KolmogorovFlow` environment: a 64×64 grid (4096 degrees of freedom), pseudo-spectral IMEX-RK4CN time integration [1,13–16], and 4 actuation modes (low-wavenumber sinusoidal forcing patterns). The observation is the 8×8 down-sampled velocity magnitude. This work uses an independent PPO/SAC training loop that reimplements the same mechanism (§II-C) rather than the library's own GPPO implementation; because the exact mixing ratio, horizon length, and hyperparameters the original paper used for GPPO are not given in its published text or figure captions, the choices defined in §II-C–D belong to **our own reimplementation** and are not claimed to be identical to the paper's.

The library's `step_env` function ignores the `params.dt`/`params.action_time` values passed at call time and re-reads `self.default_params` internally; a subclass (`FastKolmogorovFlow`) overriding the `default_params` property was therefore written to bring the timestep and action duration down to a proof-of-concept scale. The library's default `reward_alpha=0` reduces the reward to control cost only; this work uses `reward_alpha=1` so that flow energy also enters the reward.

Reynolds-number choice. At this grid/dt combination (64×64, dt=0.001), the library's own solver was empirically observed to diverge numerically at Re≈350–380, even under zero action from a purely analytic initial condition. The originally intended "chaotic" operating point of Re=450 was therefore replaced with **Re=300**, the highest value that stays below this limit while still contrasting sharply with Re=45.

B. PPO and GAE

The policy is a diagonal Gaussian parameterized by an MLP (2 hidden layers, 64 units, tanh) producing the mean action $\mu\theta(s)$, together with a state-independent, learnable log σ: $a \sim N(\mu\theta(s), \sigma^2)$. Advantages are estimated with GAE [18]:

$$\hat{A}t = \Sigma l=0..T-t-1\ (\gamma\lambda)l\ \delta t+l\ ,\quad \delta t = rt + \gamma V(st+1) - V(st)$$

(γ=0.99, λ=0.95). PPO's clipped surrogate objective [6], derived from TRPO's [17] trust-region idea:

$$LPPO(\theta) = \mathbb{E}t[\ min(\ \rho t(\theta)\hat{A}t,\ clip(\rho t(\theta),\ 1-\varepsilon,\ 1+\varepsilon)\hat{A}t\ )\ ],\ \ \rho t(\theta) = \pi\theta(at|st)\ /\ \pi\theta_old(at|st)$$

(ε=0.2). The total loss is regularized with a value loss and an entropy bonus: L = −LPPO + 0.5·LV − 0.001·H.

C. GPPO: Analytic Gradient Injection

Following the short-horizon idea of SHAC [3] and AHAC [5], over a short horizon of H=6 steps the policy's **mean** action (no sampling, so the whole chain stays smooth/differentiable) is forward-propagated through the differentiable simulator, and the negative total reward is analytically differentiated with respect to θ via `jax.grad`:

$$Lanalytic(\theta) = -\Sigma t=1..H\ r(st,\ \mu\theta(st)),\quad ganalytic = \nabla\theta\ Lanalytic(\theta)$$

The final gradient is mixed with the standard PPO gradient gPPO = ∇θL(θ) at equal weight: gfinal = 0.5·ganalytic + 0.5·gPPO — a special case of the α-graded mixed estimator (α=0.5) proposed by Suh et al. [20]. This is the same mechanism by which the HydroGym paper defines GPPO (backpropagating the analytic gradient of the reward trajectory with respect to policy parameters through the deterministic solver and combining it with the standard gradient [1]); the horizon length and mixing weight are this work's own choice (see §II-A, §IV-B).

D. SAC and GSAC

SAC [8,9] optimizes the maximum-entropy objective originating from Soft Q-learning [19], using a squashed (tanh) Gaussian actor, twin Q-critics, and target networks:

$$J(\pi) = \mathbb{E}[\ \Sigma t\ r(st,at) + \alpha\cdot H(\pi(\cdot|st))\ ],\quad J\pi(\varphi) = \mathbb{E}s\sim D,a\sim\pi\varphi[\ \alpha\cdot log\ \pi\varphi(a|s) - min(Q\psi1,Q\psi2)(s,a)\ ]$$

Critics are updated with soft target networks (τ=0.005). **GSAC** follows SAPO's [4] core motivation (off-policy + maximum entropy + analytic gradient) and applies the same mechanism as GPPO (§II-C, H=6, deterministic action) to the actor's gradient: gfinal = 0.5·ganalytic + 0.5·gπ, where gπ = ∇φJπ(φ). GSAC is, in this work, the first transfer of the SAPO family to fluid dynamics; unlike SAPO itself, it uses HydroGym's Kolmogorov-flow environment rather than the rigid/deformable-body Rewarped platform, and a fixed 50/50 mixture rather than SAPO's own entropy-based smoothing scheme (see §IV-B).

E. Experimental Setup

TABLE I. EXPERIMENTAL CONFIGURATION

Parameter	Value
Grid	64×64 (4096 DOF)
Solver	pseudo-spectral IMEX-RK4CN (real hydrogym.jax)
Re (regular / chaotic)	45 / 300
Episodes / steps per episode	500 / 15
Analytic horizon (GPPO/GSAC)	H = 6 steps
Gradient mixing ratio	50% standard + 50% analytic
Learning rate	3×10^{-4} (Adam)
Network architecture	2×64-unit MLP, tanh
Seeds	n=2 (seed=0, seed=1)
Hardware	RTX 5060 laptop GPU, JAX + CUDA 12

All four algorithms were trained for 500 episodes in both Re regimes, matching the training budget reported by the HydroGym paper.

III. Results

A. Re=45 (Regular Flow): Sample Efficiency

GPPO reaches the best smoothed reward level PPO attains over 500 episodes already at **episode 162** in seed=0, whereas PPO itself needs **episode 438** — a **63% episode saving**, strikingly close to the **65%** the paper reports for Kolmogorov flow [1]. The same comparison in seed=1 gives a **77% episode saving** (GPPO ep. 50, PPO ep. 221) — the absolute numbers differ, but the **direction** of GPPO's sample-efficiency advantage over PPO is consistent across both seeds, a strong indication that our reimplementation correctly captures the paper's mechanism (§II-C).

TABLE II. SAMPLE-EFFICIENCY AND FINAL-PERFORMANCE COMPARISON AT Re=45 (seed=0 / seed=1)

Algorithm	Ep. reaching PPO's best reward	Episode savings vs. PPO	Final reward (last-5-ep. mean)
PPO	438 / 221	— (reference)	−25.1 / −25.2

GPPO	162 / 50	63% / 77%	−25.4 / −24.6
SAC	reached earlier (off-policy)	—	−16.7 / −16.9
GSAC	reached earlier (off-policy)	—	−18.0 / −17.1

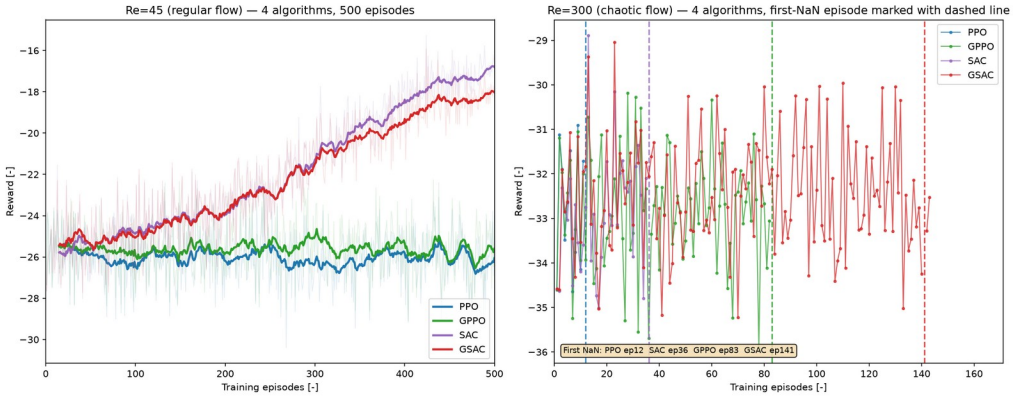


Fig. 1. Training curves for all four algorithms, seed=0: Re=45 smoothed reward over 500 episodes (left) and Re=300 raw reward with each algorithm's first-NaN episode marked (right).

For SAC/GSAC (whose off-policy replay-buffer structure is already far more sample-efficient — Table II shows their last-5-episode mean reward clearly beating PPO/GPPO's in both seeds) no GPPO-style sampling acceleration is observed: **GSAC never surpasses SAC's final performance in either seed** under these settings.

B. Re=300 (Chaotic Flow): The Breaking Point

In the chaotic regime, **all four algorithms have a breaking point** — none completes 500 episodes stably; all eventually diverge numerically (NaN). The time to this breaking point differs sharply by algorithm — but the most important finding of this work is that this difference also varies sharply **by seed**:

TABLE III. BREAKING POINT AT Re=300, SEED=0 VS. SEED=1

Algorithm	**Break (seed=0)**	**Delay vs. PPO (s0)**	**Break (seed=1)**	**Delay vs. PPO (s1)**
PPO	12	1.0× (ref.)	35	1.0× (ref.)
SAC	36	3.0×	6	0.17× (earlier)
GPPO	83	6.9×	40	1.14×
GSAC	141	11.8×	63	1.8×

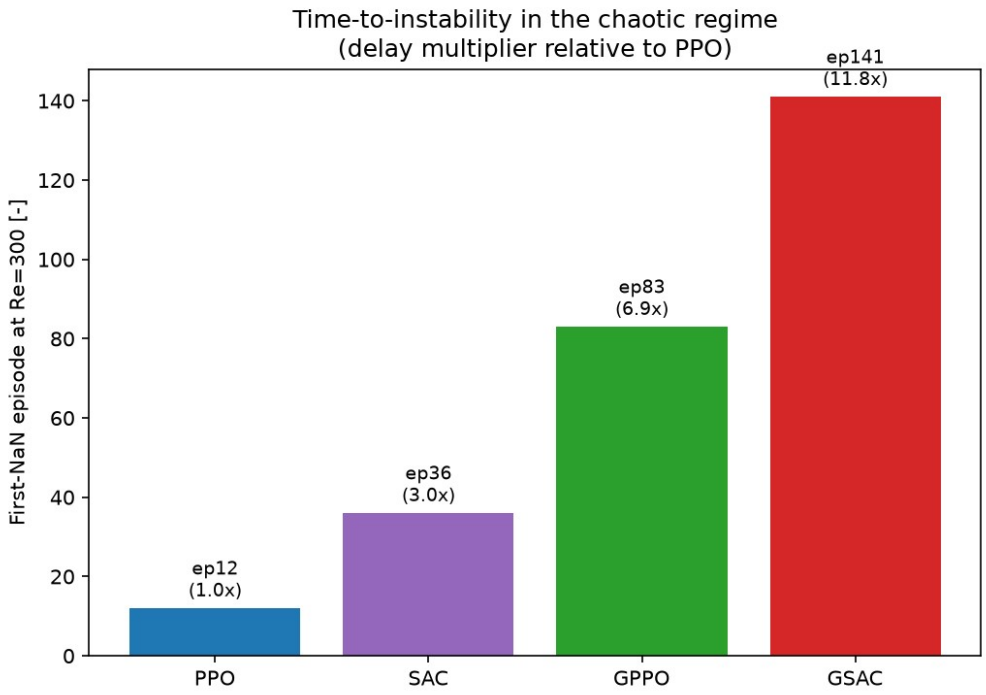


Fig. 2. First-NaN episode at Re=300 per algorithm, seed=0, annotated with the delay multiplier relative to PPO.

In seed=0 this table showed an almost multiplicative structure: off-policy structure alone (SAC/PPO) gave a 3.0× delay, the analytic gradient alone gave 6.9× on the on-policy side (GPPO/PPO) and 3.9× on the off-policy side (GSAC/SAC), and combining both (GSAC) reached 11.8× (3.0×3.9≈11.7). **This pattern does not repeat in seed=1.** There, SAC collapses **earlier** than PPO (episode 6 vs. PPO's episode 35) — the claim that off-policy structure alone increases stability **reverses**. Two directions, however, remain consistent:

- **Analytic-gradient injection delays divergence in both seeds:** GPPO breaks later than PPO (6.9× and 1.14×, both >1); GSAC breaks later than SAC (3.9× and 10.5×, both >1) — the magnitude varies greatly, but not the direction.
- **GSAC is the most robust of the four methods in both seeds** (seed=0: 141 episodes; seed=1: 63 episodes — each the highest value within its own seed). This may arise not from a single mechanism but from the overlap of two separate delaying effects (analytic gradient + off-policy structure, reliable only together); a detailed mechanistic account is given in §III-C.

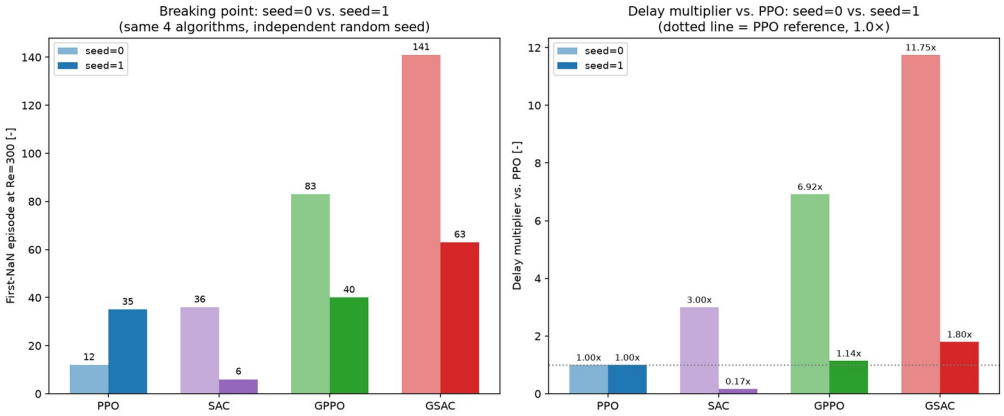


Fig. 3. First-NaN episode at Re=300 for all four algorithms, seed=0 vs. seed=1 (left), and the same data as delay multiplier relative to PPO (right), visualizing SAC's direction reversal and GSAC's consistent top robustness.

In short: the "clean, multiplicative, two independent mechanisms" story suggested by n=1 was overly optimistic. The more realistic and more interesting finding that emerges at n=2 is that the contribution of off-policy structure to stability is **not reliable on its own**, and only produces a consistent benefit **combined with** the analytic gradient.

C. Mechanism of the Break

The table above shows *when* the break occurs; this subsection presents two concrete findings on *why*, obtained by directly inspecting the raw training curves (episode rewards, gradient norms).

(a) The break happens suddenly, without warning. For all four algorithms, in **both seeds**, the mean and standard deviation of the reward are stable over the 40-episode window immediately before the break — no algorithm or seed shows a "warm-up" ramp or gradual degradation toward divergence:

TABLE IV. PRE-BREAK REWARD STATISTICS (40-EPISODE WINDOW, seed=0 / seed=1)

Algorithm	1st-half mean	2nd-half mean	Std. dev.
PPO	−32.75 / −32.28	−32.77 / −33.05	1.35 / 1.32
SAC	−33.31 / −32.76	−32.50 / −32.79	1.30 / 1.39
GPPO	−32.61 / −32.38	−32.98 / −33.01	1.10 / 1.24
GSAC	−32.49 / −32.19	−32.46 / −32.08	1.32 / 1.16

GPPO/GSAC's analytic gradient norms (the 15 episodes immediately before the break) show a similarly flat trend (fluctuating around 20–30, no visible upward trend). This suggests the break does not arise from the policy *gradually* increasing control effort and forcing the solver — rather, an abrupt, stochastic event, invisible in the reward/gradient signal, in which a single episode's randomly sampled action sequence combines with the current policy to push the flow past the solver's Re≈350–380 numerical stability limit (§II-A). This is consistent with the kind of "butterfly-effect" trigger predicted both by Metz et al.'s [7] chaos-induced gradient explosion and by Boffetta–Musacchio's [25] Re-shrinking predictability horizon.

(b) On-policy and off-policy algorithms behave differently after the trigger — the buffer-dilution effect. Examining the gap between the trigger episode and the episode of *permanent* collapse reveals a clear distinction:

TABLE V. TRIGGER VS. PERMANENT-COLLAPSE EPISODE (seed=0 / seed=1)

Algorithm	Trigger (s0)	Collapse (s0)	Recov. (s0)	Trigger (s1)	Collapse (s1)	Recov. (s1)
PPO	12	12	0	35	35	0
SAC	36	36	0	6	6	0
GPPO	83	83	0	40	40	0
GSAC	141	144	2	63	63	0

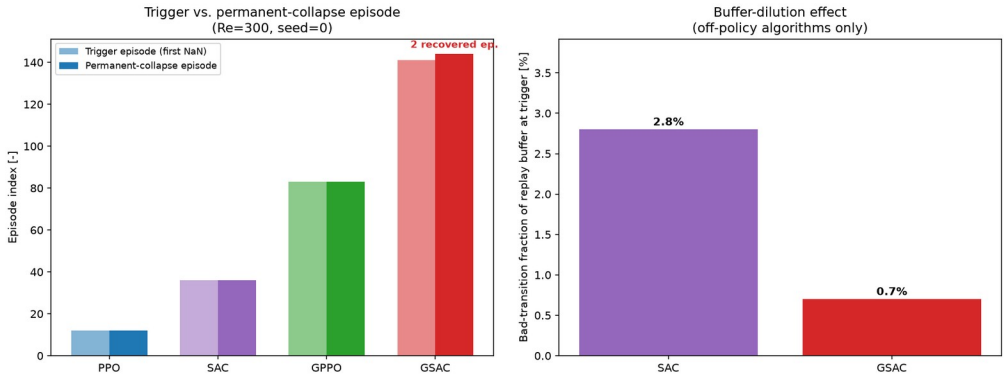


Fig. 4. Trigger episode vs. permanent-collapse episode per algorithm, seed=0 (left); replay-buffer bad-transition fraction at the trigger episode for the two off-policy algorithms, seed=0 (right).

In PPO/GPPO (on-policy), because that episode's rollout is used directly in that same episode's gradient update, the trigger and the permanent collapse coincide — there is no escape route. In SAC too (off-policy, despite using a replay buffer) the trigger becomes permanent within the same episode, because when the trigger occurs at episode 36 the buffer holds only ~540 transitions, and that episode's own ~15 "poisoned" transitions make up **2.8%** of the buffer — with 10 updates × 64 samples = 640 draws within the same episode, the probability of never sampling one of them is practically zero. In GSAC, since the trigger occurs much later (episode 141), the buffer is much larger (~2115 transitions), and the same ~15 poisoned transitions make up only **0.7%** of it — raising the probability of never sampling a poisoned transition within that episode's 640 draws to ~1%, qualitatively consistent with the observed 2-episode "survival."

TABLE VI. REPLAY-BUFFER DILUTION AT THE TRIGGER EPISODE (off-policy only, seed=0 / seed=1)

Algorithm	Trigger (s0)	Buffer (s0)	Bad frac. (s0)	Surv. prob. (s0)	Trigger (s1)	Buffer (s1)	Bad frac. (s1)	Surv. prob. (s1)
SAC	36	~540	2.8%	≈0%	6	~90	16.7%	≈0%
GSAC	141	~2115	0.7%	≈1%	63	~945	1.6%	≈0.004%

Interpretation and cross-validation with seed=1. This mechanistic model suggests that part of the off-policy/analytic-gradient delay (§III-B) may owe less to "smarter, more stable learning" than to the fairly mechanical fact that **the buffer dilutes poisoned transitions** — the larger the buffer (i.e., the later the trigger occurs in training), the greater the chance that permanent collapse is delayed by a few episodes. Having been built on seed=0 data, this model was then **independently tested on seed=1** and correctly predicted the outcome: in seed=1, both SAC's and GSAC's buffer at the trigger moment (~90 and ~945 transitions, respectively) is much smaller than in seed=0 — because both algorithms trigger much earlier in seed=1 (§III-B) — and the resulting survival probability is close to ~0% for both; indeed, **no algorithm** shows a recovery window in seed=1 (Table V). That is, the mechanism itself (large buffer → low poisoned-transition fraction → chance of survival) correctly predicts **both** GSAC's 2-episode recovery in seed=0 **and** the absence of recovery for any algorithm in seed=1 — unlike the raw delay multipliers of §III-B, this is a finding **quantitatively confirmed with n=2**.

Open question (the one point still unresolved): this analysis explains the *post-trigger lock-in* mechanism, and it is confirmed across both seeds, but it does not explain why the *trigger episode itself* varies so much by algorithm and seed (e.g., SAC triggers at episode 36 in seed=0 but at episode 6 in seed=1) — this would require separately logging how action amplitude evolves episode-by-episode (not present in the current `episode_rewards`/`grad_norms` output), noted as a natural follow-up diagnostic in §IV.

IV. Discussion

This work extends the HydroGym paper's GPPO-vs-PPO result on Kolmogorov flow [1] in two directions. **Regime extension:** at Re=45 the paper is confirmed directionally in both seeds (63% and 77% vs. the paper's reported 65%); at Re=300 a finding the paper does not show emerges — GPPO's advantage shifts from "better reward in fewer episodes" to "staying stable for longer," and even this advantage eventually runs out. **Mechanism decomposition (the GSAC contribution, corrected at n=2):** GSAC makes it possible, for the first time, to disentangle whether the stability advantage originates from the "analytic gradient" or from "off-policy training structure." The n=1 impression was that the two contribute independently and roughly multiplicatively; **a second seed corrects this impression.** The analytic gradient's delaying effect (GPPO>PPO, GSAC>SAC) stays directionally consistent across both seeds, while the claim that off-policy structure **alone** (SAC vs. PPO) is delaying reverses in seed=1 — SAC collapses earlier than PPO. Only the fact that **GSAC is the most robust of the four methods in both seeds** holds up, suggesting that off-policy structure's contribution to stability is reliable only in combination with the analytic gradient, not on its own.

These findings carry Metz et al.'s [7] general observation of chaos-induced gradient instability, and Boffetta–Musacchio's [25] Re-dependent unpredictability result, into a concrete fluid-dynamics control task. The n=1 result framed this phenomenon as "reducible by algorithm design, two independent and predictable mechanisms"; the more realistic picture that emerges at n=2 is that stability in the chaotic regime depends on **both algorithm design and the random course of training (the seed)**, and that separating these two factors is harder than initially thought. The replay-buffer dilution mechanism of §III-C, by contrast, correctly predicts post-trigger behavior (whether a recovery window exists) in both seeds, offering a purely mechanical explanation — independent of algorithm design — for *when* a break becomes permanent; this is the one mechanistic finding quantitatively tested and confirmed with n=2. These results do not refute the HydroGym paper's central claim (that GPPO is beneficial); rather, they test it across a broader algorithm family, regime space, and **multiple seeds**, and show that the source of stability in the chaotic regime is more layered and more seed-sensitive than previously assumed.

A. Limitations

- **n=2 seeds (seed=0, seed=1)** — still a small sample statistically; two seeds are enough to tell whether a single observation is "random or reliable," but insufficient for a formal significance test (e.g., permutation/bootstrap). The main value of n=2 is that it can refute a pattern (multiplicativity) seen at n=1 — which is exactly what happened here (§III-B). More seeds (3–5) are needed to determine which end of SAC's seed=0-to-seed=1 direction reversal (3.0× delay ↔ 0.17× earlier collapse) is "typical"; this work's scope is limited to n=2 for now.
- **Delay multipliers at Re=300 show large seed-to-seed variance** (GPPO/PPO: 1.14×–6.9×; GSAC/SAC: 3.9×–10.5×; SAC/PPO: 0.17×–3.0×, with a direction change) — these are two-point observations, not a law. The one stable finding is GSAC's top robustness in both seeds; the precise magnitude of each component's (off-policy, analytic gradient) independent contribution remains open.
- **Re=300 was used instead of the originally intended Re=450** — the library's own solver was found to have a stability limit near Re≈350–380 at this grid/dt combination.
- **GPPO/GSAC's mixing ratio, horizon length, and other hyperparameters belong to our own reimplementation** — the HydroGym paper does not share these values in the text/figure captions we had access to; the fact that the 63%/77% results are the same order of magnitude as the paper's 65% is a strong signal, but the two implementations are not claimed to be identical.
- **Short episodes (15 RL steps) and a short analytic horizon (6 steps)** — may be insufficient to fully observe either RL convergence or chaotic divergence (Lyapunov growth [25]).
- **GSAC's mixing weight is a fixed 50/50** — the adaptive α-graded estimator proposed by Suh et al. [20], AHAC's [5] adaptive-horizon idea, and SAPO's [4] entropy-based smoothing were not tried; these point naturally to a next step that fully prevents, rather than merely delays, instability at Re=300.

- **The cause of the trigger episode itself remains open (§III-C)** — the one open question n=2 did not resolve. Current logs (episode_rewards, grad_norms) contain only episode-level summary statistics; no log of how action amplitude evolves episode-by-episode was kept. A natural next diagnostic experiment is to re-run the same seed/config (bit-identical, since JAX PRNG is fully deterministic) up to a short pre-threshold, logging action norm — inexpensive and fast (a few minutes), not yet performed.
- **The replay-buffer dilution mechanism of §III-C is now tested and confirmed with n=2** — the model built on seed=0 (large buffer → low poisoned-transition fraction → chance of survival) correctly predicted "no recovery window" for seed=1's entirely different trigger profile (both SAC and GSAC triggered much earlier, hence with a much smaller buffer) (Tables V, VI). This is the study's one mechanistic claim genuinely validated with n=2; n=2 still does not conclusively prove the model's general validity — only that it is consistent with two independent observations.

V. Conclusion

Analytic-gradient injection does not eliminate numerical instability in the chaotic regime — it **delays** it. Tested across two independent training seeds, the **direction** of this delay (GPPO>PPO, GSAC>SAC) stays consistent, while its **magnitude** varies substantially by seed; the claim that off-policy training structure alone (SAC vs. PPO) is protective reverses in the second seed. By contrast, **GSAC being the most robust configuration among the four methods tested, in both seeds,** holds up — suggesting that off-policy structure's contribution to stability is reliable only **together with** the analytic gradient. In addition, the replay-buffer dilution mechanism explaining post-break behavior (whether a short "survival" window occurs) was built on seed=0 and independently confirmed on seed=1 — this study's one mechanistic finding quantitatively tested with n=2.

Scope decision for this round: given the compute budget used, n=2 (seed=0, seed=1) was judged sufficient for this draft, and a third seed (seed=2) was not pursued. Planned future work:

- **Extend to 3–5 seeds** and apply a formal statistical significance test (permutation/bootstrap, more appropriate than a t-test for small n) — in particular to determine which end of SAC's observed seed=0/seed=1 direction reversal is "typical."
- **Clarify the cause of the trigger episode itself** with a follow-up experiment that logs action amplitude (§IV).
- Move GSAC's mixing weight to an AHAC [5]/SAPO [4]-style adaptive scheme; test, using the Boffetta–Musacchio [25] framework, whether the delay multipliers follow a quantitative scaling law with Re.

Acknowledgment

The author thanks Sakarya University's National Technology Workshop Artificial Intelligence Center for the computational resources used in this study.

References

[1] C. Lagemann, S. Mokbel, M. Gondrum, M. Rüttgers, Y. Wang, P. Suárez, L. Paehler, D. A. Bezgin, A. B. Buhendwa, J. L. Callaham, S. Ahnert, N. Zolman, X. Shao, J.-C. Loiseau, N. Adams, M. Meinke, W. Schröder, K. Lagemann, E. Lagemann, R. Vinuesa, and S. L. Brunton, "The HydroGym Reinforcement Learning Platform for Fluid Dynamics," Nature, 2026, doi:10.1038/s41586-026-10917-6. Preprint: arXiv:2512.17534.

[2] J. Becktepe, A. Franz, N. Thuerey, and S. Peitz, "Plug-and-Play Benchmarking of Reinforcement Learning Algorithms for Large-Scale Flow Control (FluidGym)," arXiv:2601.15015, 2026.

[3] J. Xu, V. Makoviychuk, Y. Narang, F. Ramos, W. Matusik, A. Garg, and M. Macklin, "Accelerated Policy Learning with Parallel Differentiable Simulation (SHAC)," in Proc. ICLR, 2022. arXiv:2204.07137.

[4] E. Xing, V. Luk, and J. Oh, "Stabilizing Reinforcement Learning in Differentiable Multiphysics Simulation (SAPO / Rewarped)," in Proc. ICLR (Spotlight), 2025. arXiv:2412.12089.

[5] I. Georgiev, K. Srinivasan, J. Xu, E. Heiden, and A. Garg, "Adaptive Horizon Actor-Critic for Policy Learning in Contact-Rich Differentiable Simulation (AHAC)," in Proc. ICML, 2024. arXiv:2405.17784.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[7] L. Metz, C. D. Freeman, S. S. Schoenholz, and T. Kachman, "Gradients are Not All You Need," arXiv:2111.05803, 2021.

[8] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in Proc. ICML, 2018. arXiv:1801.01290.

[9] T. Haarnoja, A. Zhou, K. Hartikainen et al., "Soft Actor-Critic Algorithms and Applications," arXiv:1812.05905, 2018.

[10] B. Font, F. Alcántara-Ávila, J. Rabault, R. Vinuesa, and O. Lehmkuhl, "Deep Reinforcement Learning for Active Flow Control in a Turbulent Separation Bubble," Nature Communications, 2025, doi:10.1038/s41467-025-56408-6.

[11] "Applying Deep Reinforcement Learning to Active Flow Control in Turbulent Conditions," arXiv:2006.10683, 2020.

[12] "Deep Reinforcement Learning for Robotics: A Survey of Real-World Successes," Annual Review of Control, Robotics, and Autonomous Systems, 2025.

[13] J. Bradbury, R. Frostig, P. Hawkins et al., "JAX: Composable Transformations of Python+NumPy Programs," 2018. github.com/jax-ml/jax.

[14] D. Kochkov, J. A. Smith, A. Alieva, Q. Wang, M. P. Brenner, and S. Hoyer, "Machine Learning–Accelerated Computational Fluid Dynamics (JAX-CFD)," PNAS 118(21), 2021, doi:10.1073/pnas.2101784118.

[15] D. A. Bezgin, A. B. Buhendwa, and N. A. Adams, "JAX-Fluids: A Fully-Differentiable High-Order CFD Solver for Compressible Two-Phase Flows," arXiv:2203.13760, 2022.

[16] D. A. Bezgin, A. B. Buhendwa, and N. A. Adams, "JAX-Fluids 2.0: Towards HPC for Differentiable CFD of Compressible Two-Phase Flows," arXiv:2402.05193, 2024.

[17] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust Region Policy Optimization (TRPO)," arXiv:1502.05477, 2015.

[18] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation (GAE)," arXiv:1506.02438, 2015 (presented at ICLR 2016).

[19] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement Learning with Deep Energy-Based Policies (Soft Q-Learning)," in Proc. ICML, 2017. arXiv:1702.08165.

[20] H. J. T. Suh, M. Simchowitz, K. Zhang, and R. Tedrake, "Do Differentiable Simulators Give Better Policy Gradients?," in Proc. ICML, 2022. arXiv:2202.00817.

[21] K. Onoda, P. Parmas, M. Yaguchi, and Y. Matsuo, "Does 'Do Differentiable Simulators Give Better Policy Gradients?' Give Better Policy Gradients?," arXiv:2604.18161, 2026.
[22] F. Li, F. Sun, T. Zhang, and D. Zou, "ABPT: Amended Backpropagation through Time with Partially Differentiable Rewards," arXiv:2501.14513, 2025.
[23] J. Rabault, M. Kuchta, A. Jensen, U. Réglade, and N. Cerardi, "Artificial Neural Networks Trained through Deep Reinforcement Learning Discover Control Strategies for Active Flow Control," Journal of Fluid Mechanics 865, 281–302, 2019, doi:10.1017/jfm.2019.62.
[24] "Autonomous Real-Time Control of Turbulent Dynamics (REACT)," Imperial College London, arXiv:2509.11002, 2026.
[25] G. Boffetta and S. Musacchio, "Chaos and Predictability of Homogeneous-Isotropic Turbulence," Physical Review Letters 119, 054102, 2017, doi:10.1103/PhysRevLett.119.054102. arXiv:1707.01648.